

Yes — and actually your situation is **very common for full-stack developers who become backend-heavy**.

You don't need to relearn React from zero. You need to **maintain React muscle memory while making backend/Python your primary interview strength**.

Since you're currently working mostly with **Python/FastAPI + SQL/NoSQL + RAG/LLM/backend systems**, I'd suggest a daily split like this:

Your daily interview preparation

Assuming you can spend **2–2.5 hours/day**:

Time	Focus	What to do
45 min	Python + Backend	Python concepts, FastAPI, SQL, APIs
30 min	React	React concepts + small coding
30 min	DSA	1–2 problems
20 min	System Design	One backend/system-design topic
15 min	Interview Revision	Speak answers aloud

The important part is **React every day, but only 30 minutes**.

1. Don't study React like a beginner

You already know React. Your problem is **recall**, not learning.

Instead of:

"Today I'll study React."

Do:

"Today I'll rebuild a small React concept from memory."

For example:

Monday — React fundamentals

Review:

- Components
- Props
- State
- Events
- Conditional rendering
- Lists

- Keys

Then write something small:

```
function UserList() {
  const [users, setUsers] = useState([]);

  return (
    <div>
      {users.map(user => (
        <div key={user.id}>
          {user.name}
        </div>
      ))}
    </div>
  );
}
```

Then ask yourself:

- Why do we need `key`?
- What happens when state changes?
- Why shouldn't I mutate `users` directly?
- Props vs state?
- What causes a component to re-render?

That's much more valuable for interviews than watching another 2-hour React course.

2. Use a weekly React rotation

I'd recommend this:

Monday — React Core

- Components
- Props
- State
- Events
- Conditional rendering
- Lists
- Forms

Tuesday — Hooks

- `useState`
- `useEffect`
- `useRef`

- `useMemo`
- `useCallback`

Especially understand:

```
useState
  ↓
state changes
  ↓
component re-renders
  ↓
useEffect may run
```

And be able to explain **when and why**.

Wednesday — Advanced React

Focus on interview-heavy topics:

- Context API
- Custom hooks
- Component lifecycle
- Controlled vs uncontrolled components
- `React.memo`
- Re-rendering
- State management
- Performance optimization

You previously studied topics like `useCallback`, `useMemo`, code splitting and lazy loading, so these should be part of your revision rather than completely new topics.

Thursday — React + API

This is particularly important for **you as a full-stack developer**.

Build:

```
React
  ↓
Axios / fetch
  ↓
FastAPI
  ↓
Service
  ↓
MySQL
```

For example:

```
Login
↓
React form
↓
POST /login
↓
FastAPI
↓
JWT
↓
React stores authentication state
↓
GET /profile
```

You should be able to explain the entire flow in an interview.

Friday — React Performance

Study:

- Re-rendering
- `React.memo`
- `useMemo`
- `useCallback`
- Lazy loading
- Code splitting
- Virtualization
- Debouncing
- Throttling
- Bundle optimization

Interview question:

"Your React page is becoming slow. How would you debug it?"

You should have a structured answer.

Saturday — Build

Don't study.

Build something for **1–2 hours**.

For example:

Admin Dashboard

- Login
- Dashboard
- Users
- Search
- Pagination
- Filters
- CRUD
- API integration

This is extremely useful because one small project can refresh dozens of React concepts.

Sunday — Interview simulation

Take 30–60 minutes and answer questions **without looking at notes**.

For example:

What is reconciliation?

What causes a React component to re-render?

`useMemo` vs `useCallback`?

Why shouldn't we use array index as a key?

Controlled vs uncontrolled components?

What is code splitting?

CSR vs SSR?

How does React handle events?

Context API vs Redux?

If you can't answer, mark it:

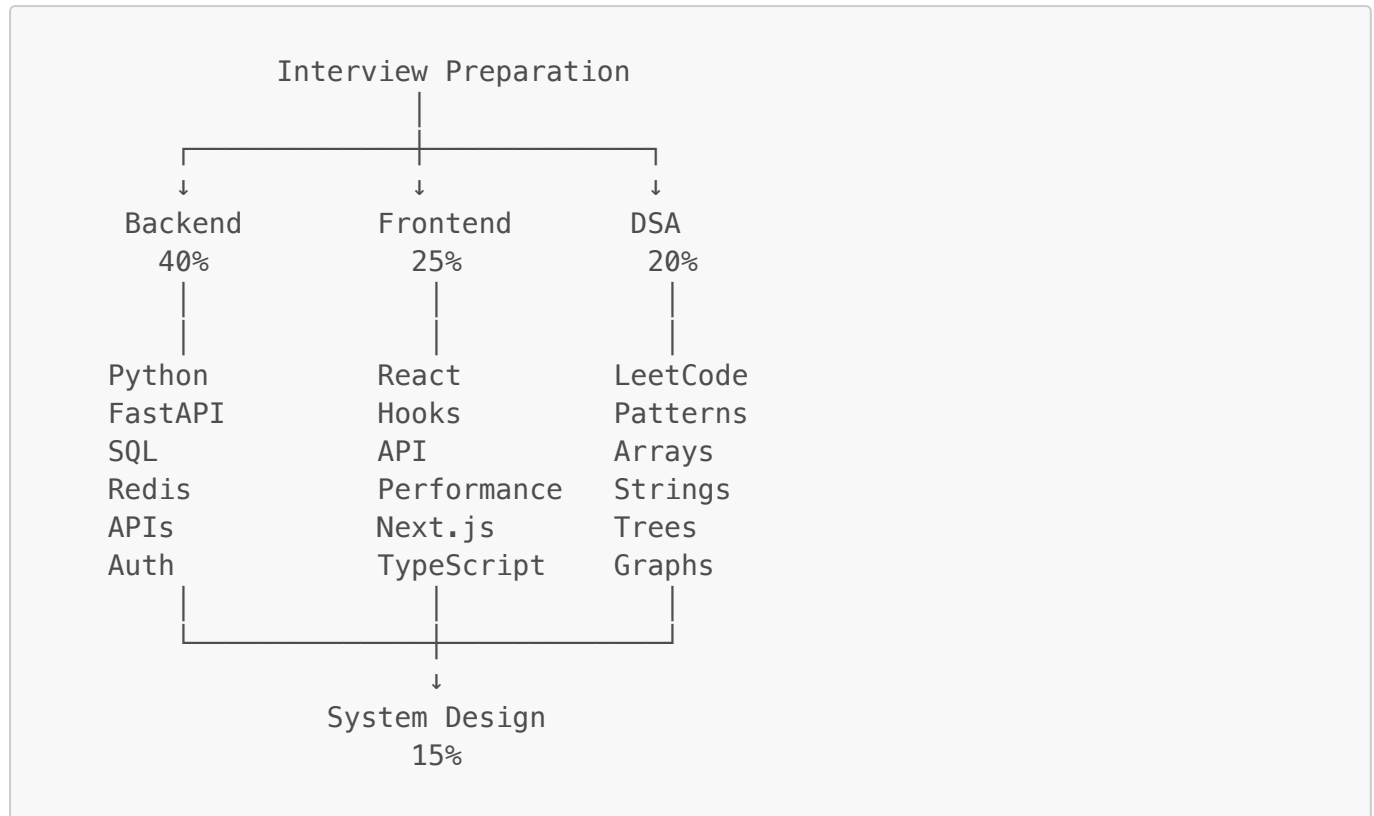
- ✗ Don't know
- ⚠ Partially know
- ✓ Can explain

Next week, focus mostly on ✗ and ⚠.

3. Your overall preparation should look like this

Because you're a **full-stack developer**, don't spend 50% of your time on React.

I'd use:



This matches your current experience better than trying to become a React specialist.

4. Your biggest advantage

Your recent backend work gives you a **very strong interview story**.

You can position yourself as:

Full-stack developer with strong backend expertise, currently working extensively with Python/FastAPI, databases, APIs and AI/RAG systems, while maintaining React expertise on the frontend.

That's much better than trying to pretend you're a 50/50 React/Backend developer.

And your backend preparation should go deeper:

Python

- Data types
- Functions
- Decorators
- Generators
- Iterators
- Context managers
- `*args`, `**kwargs`
- Exception handling

- Async/await
- GIL
- Multiprocessing vs multithreading
- Memory management
- Python internals

FastAPI

- Dependency injection
- Middleware
- Authentication
- Pydantic
- Async endpoints
- Background tasks
- Exception handling
- Validation
- Pagination
- API architecture
- Service/repository patterns

Database

Since you've been working with MySQL and MongoDB:

- Joins
- Indexes
- Transactions
- Isolation levels
- Query optimization
- Normalization
- Locks
- Pagination
- SQLAlchemy
- MongoDB indexing
- Aggregation

System Design

Then connect all of this:

```
Client
  ↓
Load Balancer
  ↓
FastAPI
  ↓
Redis
  ↓
Database
```

```
↓  
Queue  
↓  
Worker  
↓  
External Services
```

This is where your real-world backend experience becomes extremely valuable.

5. One rule I strongly recommend

Don't spend your entire preparation time consuming content.

Use this ratio:

```
30% Learning  
70% Recall + Coding + Explaining
```

For React especially:

Bad

```
Watch React course for 2 hours  
↓  
Feel like you understand React  
↓  
Forget it after 2 weeks
```

Better

```
10 min → revise concept  
10 min → write code  
10 min → answer interview questions
```

That 30-minute React session is enough to keep React alive in your memory.

6. I would structure your preparation as a 30-day cycle

Week 1

React fundamentals + Python fundamentals + SQL

Week 2

React hooks + FastAPI + API design

Week 3

React performance + advanced Python + database optimization

Week 4

Full-stack system design + mock interviews + DSA revision

Then repeat the cycle with harder questions.

Most important: make it sustainable

Don't try to study:

Python + React + Node + SQL + DSA + System Design + AI + Cloud

for 6 hours every day.

You'll burn out.

Instead:

Every day

```
45 min → Backend
30 min → React
30 min → DSA
20 min → System Design
15 min → Speak interview answers
```

Even **2 hours consistently for 3 months** will make a huge difference.

And for you specifically, I would make **Python/FastAPI + SQL + system design your strongest area**, while using the daily React practice to prevent frontend knowledge from fading.